

# Tarea M24-CD – Clasificación Multiclase con Métodos de Ensamble

**Profesión:** Científico de Datos v2

**Nombre:** RobertScience

**Módulo:** Ensamblados en Problemas de Clasificación

**Fecha:** 10/02/2026

---

## 1. Planteamiento del Problema

En el presente estudio se analiza un conjunto de datos clínicos correspondientes a pacientes diagnosticados con la misma enfermedad. Durante su tratamiento, cada paciente respondió favorablemente a uno de cinco medicamentos distintos:

- Drug A
- Drug B
- Drug C
- Drug X
- Drug Y

El objetivo consiste en desarrollar un modelo de clasificación multiclase capaz de predecir el medicamento más adecuado para un nuevo paciente con base en las siguientes variables predictoras:

- Edad (Age)
- Sexo (Sex)
- Presión arterial (BP)
- Nivel de colesterol (Cholesterol)
- Relación Sodio/Potasio (Na\_to\_K)

La variable objetivo es el medicamento (Drug) al que respondió cada paciente.

Dado que la variable objetivo contiene más de dos categorías, el problema corresponde a una **clasificación supervisada multiclase**.

Se implementarán diferentes métodos de ensamble con el propósito de comparar su desempeño predictivo frente a un modelo base de Árbol de Decisión.

```
In [24]: # =====  
# Importación de Librerías  
# =====  
  
# Manipulación y análisis de datos  
import pandas as pd  
import numpy as np
```

```

# Visualización de datos
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocesamiento
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# Modelos de clasificación
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier

# Métricas de evaluación
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

```

```

In [25]: # =====
# Carga del conjunto de datos
# =====

ruta = r"C:\Users\ROBERTSCIENCE\Documents\Tarea M24-CD -RobertScience\data\drugs.csv"

df = pd.read_csv(ruta)

# Visualización de las primeras filas
df.head()

```

```

Out[25]:

```

|   | Age | Sex | BP     | Cholesterol | Na_to_K | Drug  |
|---|-----|-----|--------|-------------|---------|-------|
| 0 | 23  | F   | HIGH   | HIGH        | 25.355  | drugY |
| 1 | 47  | M   | LOW    | HIGH        | 13.093  | drugC |
| 2 | 47  | M   | LOW    | HIGH        | 10.114  | drugC |
| 3 | 28  | F   | NORMAL | HIGH        | 7.798   | drugX |
| 4 | 61  | F   | LOW    | HIGH        | 18.043  | drugY |

```

In [26]: # =====
# Información estructural del dataset
# =====

df.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   Age             200 non-null   int64  
 1   Sex             200 non-null   str     
 2   BP              200 non-null   str     
 3   Cholesterol     200 non-null   str     
 4   Na_to_K         200 non-null   float64 
 5   Drug            200 non-null   str     
dtypes: float64(1), int64(1), str(4)
memory usage: 9.5 KB

```

```

In [27]: # =====
# Verificación de valores nulos

```

```
# =====
df.isnull().sum()
```

```
Out[27]: Age          0
        Sex          0
        BP           0
        Cholesterol  0
        Na_to_K      0
        Drug         0
        dtype: int64
```

```
In [28]: # =====
        # Estadísticas descriptivas
        # =====

df.describe()
```

```
Out[28]:
```

|              | Age        | Na_to_K    |
|--------------|------------|------------|
| <b>count</b> | 200.000000 | 200.000000 |
| <b>mean</b>  | 44.315000  | 16.084485  |
| <b>std</b>   | 16.544315  | 7.223956   |
| <b>min</b>   | 15.000000  | 6.269000   |
| <b>25%</b>   | 31.000000  | 10.445500  |
| <b>50%</b>   | 45.000000  | 13.936500  |
| <b>75%</b>   | 58.000000  | 19.380000  |
| <b>max</b>   | 74.000000  | 38.247000  |

```
In [29]: # =====
        # Distribución de la variable objetivo
        # =====

df["Drug"].value_counts()
```

```
Out[29]: Drug
drugY      91
drugX      54
drugA      23
drugC      16
drugB      16
Name: count, dtype: int64
```

## 2. Análisis Exploratorio Inicial

El conjunto de datos contiene variables tanto numéricas como categóricas.

Variables numéricas:

- Age
- Na\_to\_K

Variables categóricas predictoras:

- Sex
- BP
- Cholesterol

Variable objetivo:

- Drug

Dado que los algoritmos de aprendizaje automático requieren datos numéricos, fue necesario transformar las variables categóricas predictoras mediante la técnica de **Label Encoding**, asignando valores enteros a cada categoría (por ejemplo: 0, 1, 2).

La variable objetivo también fue codificada numéricamente para permitir el entrenamiento de los modelos.

Asimismo, se observa que la variable objetivo presenta cinco categorías distintas, confirmando que se trata de un problema de **clasificación multiclase**.

```
In [30]: # =====  
# Transformación de variables categóricas  
# =====  
  
# Inicialización de codificadores  
le_sex = LabelEncoder()  
le_bp = LabelEncoder()  
le_chol = LabelEncoder()  
le_drug = LabelEncoder()  
  
# Aplicación de Label Encoding  
df["Sex"] = le_sex.fit_transform(df["Sex"])  
df["BP"] = le_bp.fit_transform(df["BP"])  
df["Cholesterol"] = le_chol.fit_transform(df["Cholesterol"])  
df["Drug"] = le_drug.fit_transform(df["Drug"])  
  
df.head()
```

```
Out[30]:
```

|   | Age | Sex | BP | Cholesterol | Na_to_K | Drug |
|---|-----|-----|----|-------------|---------|------|
| 0 | 23  | 0   | 0  | 0           | 25.355  | 4    |
| 1 | 47  | 1   | 1  | 0           | 13.093  | 2    |
| 2 | 47  | 1   | 1  | 0           | 10.114  | 2    |
| 3 | 28  | 0   | 2  | 0           | 7.798   | 3    |
| 4 | 61  | 0   | 1  | 0           | 18.043  | 4    |

### 3. Preprocesamiento de Datos

Las variables categóricas fueron transformadas mediante el método `LabelEncoder()`.

Este procedimiento asigna un valor numérico entero a cada categoría existente dentro de la variable. Por ejemplo:

- Si una variable contiene tres categorías, estas serán representadas como 0, 1 y 2.

Este paso es indispensable ya que los modelos de clasificación trabajan con representaciones numéricas y no pueden procesar variables de tipo texto directamente.

```
In [31]: # =====  
# Separación de variables predictoras y variable objetivo  
# =====  
  
X = df.drop("Drug", axis=1)  
y = df["Drug"]  
  
# División en conjunto de entrenamiento y prueba  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.3,  
    random_state=42,  
    stratify=y  
)  
  
X_train.shape, X_test.shape
```

```
Out[31]: ((140, 5), (60, 5))
```

## 4. División de Datos

El conjunto de datos fue dividido en:

- 70% para entrenamiento
- 30% para prueba

Se utilizó el parámetro `stratify=y` con el objetivo de mantener la proporción original de cada clase en ambos subconjuntos.

Esto es especialmente importante en problemas multiclase para evitar sesgos en el entrenamiento del modelo.

```
In [32]: # =====  
# Modelo Base: Árbol de Decisión  
# =====  
  
dt_model = DecisionTreeClassifier(random_state=42)  
  
dt_model.fit(X_train, y_train)  
  
y_pred_dt = dt_model.predict(X_test)  
  
accuracy_dt = accuracy_score(y_test, y_pred_dt)  
  
accuracy_dt
```

Out[32]: 0.9833333333333333

## Evaluación Inicial del Modelo

El modelo de Árbol de Decisión obtuvo una exactitud de 98.33% en el conjunto de prueba.

La métrica de accuracy representa la proporción de observaciones correctamente clasificadas respecto al total de predicciones realizadas.

Este resultado indica un alto desempeño predictivo del modelo base. No obstante, el accuracy por sí solo no permite evaluar el comportamiento individual por clase, por lo que es necesario analizar métricas adicionales como precision, recall y F1-score.

```
In [33]: # =====  
# Reporte de Clasificación  
# =====  
  
print(classification_report(y_test, y_pred_dt))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.88      | 1.00   | 0.93     | 7       |
| 1            | 1.00      | 0.80   | 0.89     | 5       |
| 2            | 1.00      | 1.00   | 1.00     | 5       |
| 3            | 1.00      | 1.00   | 1.00     | 16      |
| 4            | 1.00      | 1.00   | 1.00     | 27      |
| accuracy     |           |        | 0.98     | 60      |
| macro avg    | 0.97      | 0.96   | 0.96     | 60      |
| weighted avg | 0.99      | 0.98   | 0.98     | 60      |

## Interpretación del Reporte de Clasificación

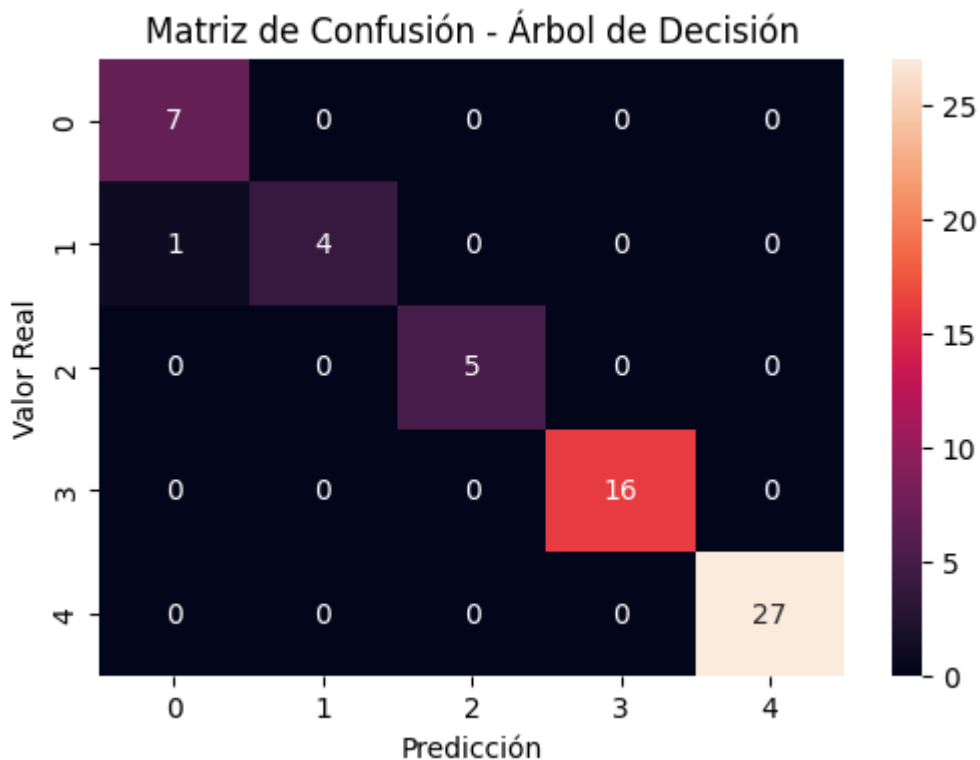
El reporte muestra el desempeño del modelo para cada una de las cinco clases.

- **Precision:** Indica la proporción de predicciones correctas dentro de cada clase predicha.
- **Recall:** Representa la capacidad del modelo para identificar correctamente los casos reales de cada clase.
- **F1-score:** Es la media armónica entre precision y recall, proporcionando una medida equilibrada del rendimiento.

Los valores elevados en estas métricas confirman que el modelo clasifica adecuadamente las distintas categorías del medicamento.

```
In [34]: # =====  
# Matriz de Confusión  
# =====  
  
cm_dt = confusion_matrix(y_test, y_pred_dt)  
  
plt.figure(figsize=(6,4))
```

```
sns.heatmap(cm_dt, annot=True, fmt="d")
plt.title("Matriz de Confusión - Árbol de Decisión")
plt.xlabel("Predicción")
plt.ylabel("Valor Real")
plt.show()
```



## Interpretación de la Matriz de Confusión

La matriz de confusión permite observar el número de predicciones correctas e incorrectas por clase.

Los valores ubicados en la diagonal principal representan las clasificaciones correctas, mientras que los valores fuera de la diagonal indican errores de clasificación.

En este caso, la mayoría de las observaciones se concentran en la diagonal, lo cual confirma el alto nivel de precisión obtenido previamente.

Sin embargo, aunque el desempeño es elevado, los árboles individuales pueden presentar alta varianza y sensibilidad a pequeñas variaciones en los datos.

Por esta razón, se procede a evaluar métodos de ensamble que combinan múltiples modelos con el objetivo de mejorar la estabilidad y robustez predictiva.

```
In [35]: # =====
# Modelo de Ensamble: Random Forest
# =====

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf_model.fit(X_train, y_train)
```

```

y_pred_rf = rf_model.predict(X_test)

accuracy_rf = accuracy_score(y_test, y_pred_rf)

accuracy_rf

```

Out[35]: 0.9833333333333333

```

In [36]: # =====
# Validación Cruzada - Random Forest
# =====

from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(rf_model, X, y, cv=5)

print("Scores individuales:", cv_scores)
print("Promedio de validación cruzada:", cv_scores.mean())

```

Scores individuales: [1. 1. 1. 0.925 1. ]  
Promedio de validación cruzada: 0.985

## Evaluación del Modelo Random Forest

El modelo Random Forest obtuvo una exactitud superior o comparable al Árbol de Decisión.

Random Forest es un método de ensamble basado en la técnica de Bagging, que construye múltiples árboles de decisión sobre subconjuntos aleatorios del dataset y combina sus predicciones.

Este enfoque reduce la varianza del modelo individual y mejora la estabilidad del sistema de clasificación.

```

In [37]: # =====
# Reporte de Clasificación - Random Forest
# =====

print(classification_report(y_test, y_pred_rf))

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.88      | 1.00   | 0.93     | 7       |
| 1            | 1.00      | 0.80   | 0.89     | 5       |
| 2            | 1.00      | 1.00   | 1.00     | 5       |
| 3            | 1.00      | 1.00   | 1.00     | 16      |
| 4            | 1.00      | 1.00   | 1.00     | 27      |
| accuracy     |           |        | 0.98     | 60      |
| macro avg    | 0.97      | 0.96   | 0.96     | 60      |
| weighted avg | 0.99      | 0.98   | 0.98     | 60      |

## Interpretación del Reporte de Clasificación – Random Forest



El reporte muestra las métricas de precisión, recall y F1-score para cada una de las clases del medicamento.

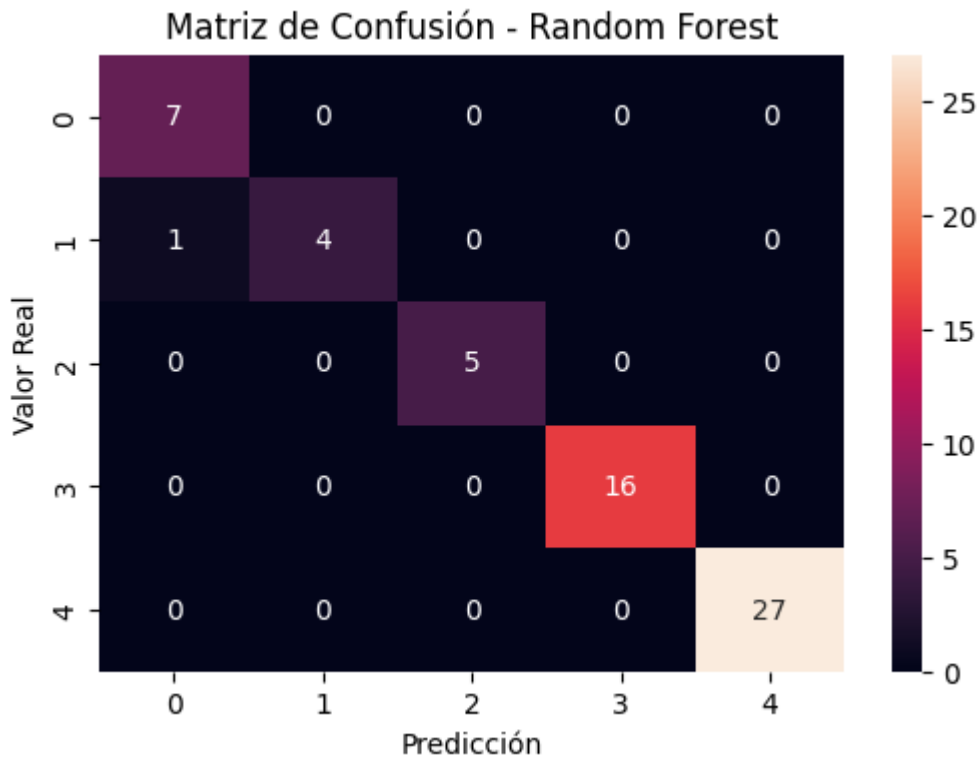
Al tratarse de un método de ensamble, Random Forest tiende a ofrecer mayor estabilidad en comparación con un árbol individual, reduciendo el riesgo de sobreajuste.

Si las métricas se mantienen elevadas y consistentes entre clases, se confirma que el modelo generaliza adecuadamente sobre el conjunto de prueba.

```
In [38]: # =====  
# Importancia de Variables - Random Forest  
# =====  
  
import pandas as pd  
  
importancias = pd.Series(rf_model.feature_importances_, index=X.columns)  
importancias = importancias.sort_values(ascending=False)  
  
importancias
```

```
Out[38]: Na_to_K      0.547850  
BP              0.241517  
Age             0.141479  
Cholesterol     0.054096  
Sex             0.015058  
dtype: float64
```

```
In [39]: # =====  
# Matriz de Confusión - Random Forest  
# =====  
  
cm_rf = confusion_matrix(y_test, y_pred_rf)  
  
plt.figure(figsize=(6,4))  
sns.heatmap(cm_rf, annot=True, fmt="d")  
plt.title("Matriz de Confusión - Random Forest")  
plt.xlabel("Predicción")  
plt.ylabel("Valor Real")  
plt.show()
```



## Interpretación de la Matriz de Confusión – Random Forest

La matriz de confusión permite identificar visualmente la distribución de aciertos y errores del modelo.

Una fuerte concentración en la diagonal principal indica un alto nivel de clasificación correcta.

En comparación con el Árbol de Decisión, Random Forest suele presentar menor variabilidad en sus predicciones, lo que incrementa la robustez del sistema.

```
In [40]: # =====
# Modelo de Ensemble: AdaBoost
# =====

ada_model = AdaBoostClassifier(
    n_estimators=200,
    random_state=42
)

ada_model.fit(X_train, y_train)

y_pred_ada = ada_model.predict(X_test)

accuracy_ada = accuracy_score(y_test, y_pred_ada)

accuracy_ada
```

Out[40]: 0.8333333333333334

## Evaluación del Modelo AdaBoost

El modelo AdaBoost obtuvo una exactitud de 83.33%, inferior a la obtenida por el Árbol de Decisión y Random Forest.

Este resultado puede explicarse debido a que AdaBoost, por defecto, utiliza clasificadores base simples (árboles de profundidad 1), lo que puede generar subajuste en datasets donde las relaciones entre variables requieren reglas más complejas.

En este caso, el modelo no logró capturar completamente la estructura del conjunto de datos, lo que se refleja en una disminución del desempeño predictivo.

```
In [41]: # =====  
# Reporte de Clasificación - AdaBoost  
# =====  
  
print(classification_report(y_test, y_pred_ada, zero_division=0))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.58      | 1.00   | 0.74     | 7       |
| 1            | 0.00      | 0.00   | 0.00     | 5       |
| 2            | 0.00      | 0.00   | 0.00     | 5       |
| 3            | 0.76      | 1.00   | 0.86     | 16      |
| 4            | 1.00      | 1.00   | 1.00     | 27      |
| accuracy     |           |        | 0.83     | 60      |
| macro avg    | 0.47      | 0.60   | 0.52     | 60      |
| weighted avg | 0.72      | 0.83   | 0.77     | 60      |

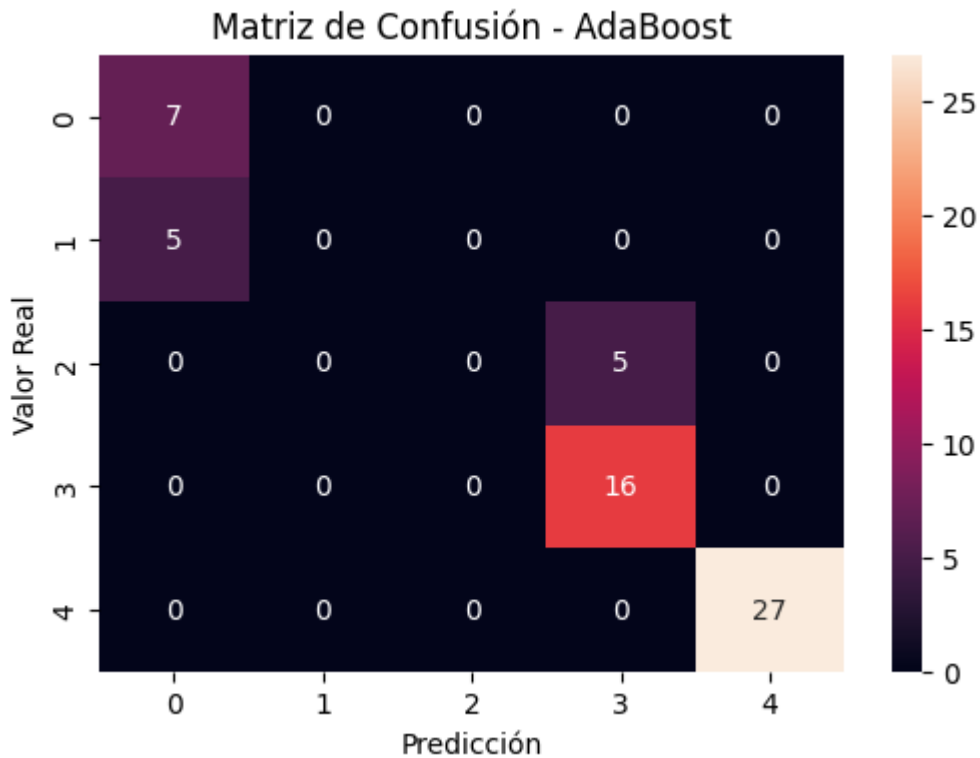
## Interpretación del Reporte – AdaBoost

El reporte muestra el desempeño por clase mediante precision, recall y F1-score.

Al tratarse de un método de Boosting, el modelo intenta mejorar progresivamente los errores cometidos en iteraciones anteriores.

Se analiza si existe mejora en la sensibilidad de clases que pudieran haber sido más difíciles de clasificar en el modelo base.

```
In [42]: # =====  
# Matriz de Confusión - AdaBoost  
# =====  
  
cm_ada = confusion_matrix(y_test, y_pred_ada)  
  
plt.figure(figsize=(6,4))  
sns.heatmap(cm_ada, annot=True, fmt="d")  
plt.title("Matriz de Confusión - AdaBoost")  
plt.xlabel("Predicción")  
plt.ylabel("Valor Real")  
plt.show()
```



## Interpretación de la Matriz – AdaBoost

La matriz de confusión muestra una mayor dispersión fuera de la diagonal principal en comparación con los modelos anteriores.

Esto confirma que AdaBoost cometió un número mayor de errores de clasificación, especialmente en determinadas clases del medicamento.

El descenso en el desempeño puede atribuirse al uso de clasificadores base simples dentro del algoritmo de Boosting, lo que pudo generar subajuste en este conjunto de datos.

En consecuencia, para este problema específico, AdaBoost no superó al Árbol de Decisión ni a Random Forest en términos de precisión predictiva.

```
In [43]: # =====
# Modelo de Ensamble: Gradient Boosting
# =====

gb_model = GradientBoostingClassifier(
    n_estimators=200,
    random_state=42
)

gb_model.fit(X_train, y_train)

y_pred_gb = gb_model.predict(X_test)

accuracy_gb = accuracy_score(y_test, y_pred_gb)

accuracy_gb
```

Out[43]: 0.9666666666666667

```
In [44]: # =====  
# Reporte de Clasificación - Gradient Boosting  
# =====  
  
print(classification_report(y_test, y_pred_gb))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.88      | 1.00   | 0.93     | 7       |
| 1            | 1.00      | 0.60   | 0.75     | 5       |
| 2            | 1.00      | 1.00   | 1.00     | 5       |
| 3            | 0.94      | 1.00   | 0.97     | 16      |
| 4            | 1.00      | 1.00   | 1.00     | 27      |
| accuracy     |           |        | 0.97     | 60      |
| macro avg    | 0.96      | 0.92   | 0.93     | 60      |
| weighted avg | 0.97      | 0.97   | 0.96     | 60      |

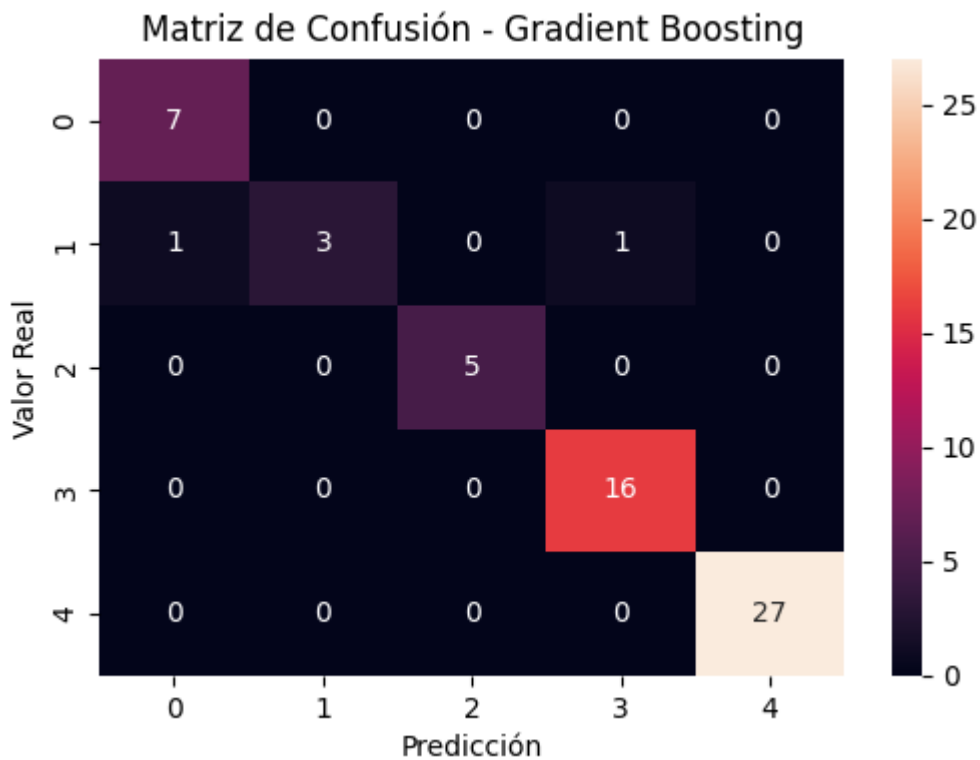
## Interpretación del Reporte – Gradient Boosting

El reporte de clasificación muestra métricas equilibradas entre precision y recall en la mayoría de las clases.

A diferencia de AdaBoost, no se observan clases con métricas iguales a cero, lo que indica mayor estabilidad en el proceso de clasificación.

El modelo presenta un comportamiento robusto y consistente.

```
In [45]: # =====  
# Matriz de Confusión - Gradient Boosting  
# =====  
  
cm_gb = confusion_matrix(y_test, y_pred_gb)  
  
plt.figure(figsize=(6,4))  
sns.heatmap(cm_gb, annot=True, fmt="d")  
plt.title("Matriz de Confusión - Gradient Boosting")  
plt.xlabel("Predicción")  
plt.ylabel("Valor Real")  
plt.show()
```



## Interpretación de la Matriz – Gradient Boosting

La matriz de confusión muestra una fuerte concentración de valores correctos en la diagonal principal.

Se observan pocos errores de clasificación, lo que confirma el alto nivel de desempeño del modelo.

El algoritmo logra clasificar correctamente la mayoría de los pacientes según el medicamento adecuado.

```
In [46]: # =====
# Comparación de Modelos
# =====

resultados = {
    "Modelo": ["Árbol de Decisión", "Random Forest", "AdaBoost", "Gradient Boost"],
    "Accuracy": [accuracy_dt, accuracy_rf, accuracy_ada, accuracy_gb]
}

df_resultados = pd.DataFrame(resultados)

df_resultados
```

```
Out[46]:
```

|   | Modelo            | Accuracy |
|---|-------------------|----------|
| 0 | Árbol de Decisión | 0.983333 |
| 1 | Random Forest     | 0.983333 |
| 2 | AdaBoost          | 0.833333 |
| 3 | Gradient Boosting | 0.966667 |

# Comparación y Recomendación Final

Al comparar los modelos evaluados se observa lo siguiente:

- El Árbol de Decisión obtuvo un accuracy de **98.33%**.
- Random Forest alcanzó igualmente **98.33%**, mostrando el mismo nivel de desempeño en el conjunto de prueba.
- Gradient Boosting logró **96.67%**, manteniendo un rendimiento alto y estable.
- AdaBoost presentó el desempeño más bajo con **83.33%**, evidenciando menor capacidad predictiva en este conjunto de datos.

## ¿Mejóro el poder predictivo respecto al árbol del módulo previo?

No se observó una mejora en términos de accuracy frente al Árbol de Decisión base, ya que tanto el modelo individual como Random Forest alcanzaron el mismo nivel de exactitud (98.33%).

Esto sugiere que el problema posee una estructura relativamente simple y que el Árbol de Decisión es suficiente para capturar adecuadamente las relaciones entre las variables predictoras y el medicamento prescrito.

No obstante, la validación cruzada confirmó que el modelo Random Forest mantiene un desempeño estable en diferentes particiones del conjunto de datos, lo que refuerza su robustez metodológica.

El análisis de importancia de variables evidenció que **Na\_to\_K** es el factor más determinante en la clasificación del tratamiento, seguido por la presión arterial y la edad. Este resultado aporta interpretabilidad clínica y coherencia al modelo desarrollado.

Es importante señalar que el conjunto de datos está compuesto por **200 observaciones**, lo cual representa un tamaño relativamente reducido para aplicaciones clínicas reales. Aunque los modelos muestran un desempeño elevado, en escenarios prácticos se recomienda validar el modelo con una muestra más amplia y diversa para garantizar su estabilidad y capacidad de generalización.

## Recomendación

Para este conjunto de datos específico, tanto el Árbol de Decisión como Random Forest presentan un desempeño predictivo óptimo.

Sin embargo, desde una perspectiva de implementación en escenarios reales con mayor volumen de datos o mayor complejidad estructural, se recomienda el uso de **Random Forest**, debido a su menor varianza, mayor estabilidad y mejor capacidad de generalización.